

SIMATS ENGINEERING

ASSESSMENT TOOL 1: AI Model Design Exercise

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO4: *Develop intelligent software agents using suitable agent architectures and learning techniques.* (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A AI Model Design Exercise

Code of Conduct:

I T Bhupal/192425243 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:*bhupalreddy*

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Dataset Selection and Data Understanding	10%	
3. Data Preprocessing and Feature Engineering	10%	
4. AI Technique Selection and Justification	10%	
5. Model Design / Architecture	10%	
6. Algorithm / Pseudocode Development	5%	
7. Model Implementation and GitHub Repository	15%	
8. Experimental Design and Results	10%	
9. Performance Evaluation and Metrics	10%	
10. Result Analysis and Interpretation	5%	
11. Limitations and Future Enhancements	5%	
12. Documentation and Technical Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

S.No.	Scenario / Problem Statement	AI Model / Technique to be Developed	Expected Development Outcome
1	Smart Loan Approval: A bank wants to automatically determine whether a customer is eligible for a loan using income, credit score, employment status, loan amount, and repayment history.	Decision Tree	Design and implement a decision-tree-based AI model and determine the most suitable attribute-selection measure such as Information Gain, Gain Ratio, or Gini Index.
2	Student Performance Prediction: A university wants to predict whether a student is likely to pass or fail a course using attendance, internal marks, assignment performance, study hours, and previous academic performance.	Inductive Learning	Construct a predictive model from training examples and derive generalized decision rules from the learned patterns.
3	Disease Risk Prediction: A healthcare system needs to classify patients into low-risk and high-risk categories based on clinical measurements.	Statistical Learning	Design a statistical learning model, train it using patient data, and evaluate its generalization ability on unseen data.
4	Image Recognition System: An organization wants to automatically classify images into categories such as vehicles, animals, and buildings.	Neural Network	Design and implement a neural-network classifier by specifying the network architecture, activation functions, loss function, optimizer, and training procedure.
5	Handwritten Character Recognition: A system must recognize handwritten characters from scanned images.	Multilayer Neural Network	Develop a multilayer neural network and demonstrate how backpropagation updates weights to minimize classification error.

6	Non-Linear Classification: A dataset contains two classes that cannot be separated using a straight-line decision boundary.	SVM with Kernel Method	Design an SVM-based classifier using an appropriate kernel function and justify the selected kernel based on the characteristics of the dataset.
7	Intelligent Recommendation Agent: An online platform wants an agent that learns which recommendations to provide based on user responses.	Reinforcement Learning	Design an RL model by defining the states, actions, rewards, policy, and learning strategy, and demonstrate how the agent learns from user feedback.
8	Autonomous Game Agent: An AI agent must learn to play a game where successful actions receive positive rewards and unsuccessful actions receive penalties.	Reinforcement Learning	Develop an RL-based game agent and demonstrate how its policy and performance improve through repeated interaction with the environment.
9	Explainable Diagnosis System: A medical AI system predicts a disease but must also provide an explanation for its prediction.	Explanation-Based Learning	Design an explanation-based learning approach using domain knowledge and training examples to generate an understandable justification for the prediction.
10	AI Model Selection Challenge: A company provides a dataset for predicting customer churn.	Comparative AI Model Design	Design and compare at least two learning approaches from decision trees, statistical learning, neural networks, or kernel methods. Select the best model based on accuracy, interpretability, computational requirements, and generalization ability.

Structure of the Report:

S.No.	Report Component	What the Student Should Include
1	Problem Statement	Clearly describe the real-world problem, input data, expected output, and AI task to be solved.
2	Objectives	State the specific goals of developing the AI model and expected outcomes.
3	Dataset Description	Provide dataset source, number of instances, features, target variable, data types, and important attributes.
4	Data Preprocessing	Explain data cleaning, missing-value handling, encoding, normalization/scaling, feature selection, and train-test splitting.
5	Selected AI Technique and Justification	Identify the selected AI technique and justify its suitability for the problem.
6	Model Design / Architecture	Present the model structure, components, input/output, parameters, hyperparameters, and architecture/flow diagram.
7	Algorithm / Pseudocode	Provide the algorithmic steps or pseudocode showing model training and prediction/decision-making.
8	Implementation	Provide programming language, libraries/tools, important code sections, and implementation details.
9	Experimental Results	Present results using tables, graphs, sample predictions, learning curves, confusion matrices, or other relevant outputs.
10	Performance Evaluation	Evaluate the model using appropriate performance metrics such as accuracy, precision, recall, F1-score, MSE, RMSE, or cumulative reward.
11	Result Analysis	Interpret results, identify important patterns, compare actual and predicted outcomes, and discuss strengths and weaknesses.
12	Limitations	Discuss limitations related to dataset, model complexity, computational resources, accuracy, generalization, or deployment.
13	Future Enhancements	Suggest improvements such as additional data, feature engineering, hyperparameter tuning, alternative algorithms, or deployment.
14	Conclusion	Summarize the problem, methodology, major findings, model performance, and overall effectiveness.
15	References	List datasets, research papers, books, websites, documentation, libraries, and other sources used.
16	GitHub Repository Link	Provide the public GitHub repository URL containing the complete source code, dataset information/source, implementation, results, README, and supporting files.

Student Performance Prediction:

1. Problem Statement

Educational institutions often struggle to proactively identify students who are at risk of failing a course until it is too late. The real-world problem is to predict whether a student is likely to pass or fail a course based on their academic and behavioral metrics.

- Input Data: Attendance percentage, internal marks, assignment scores, weekly study hours, and previous semester GPA.
- Expected Output: A binary classification indicating the final status: Pass (1) or Fail (0).
- AI Task: To solve this as a supervised binary classification problem using Inductive Learning to generate readable decision rules.

2. Objectives

- To develop an AI model capable of accurately predicting student success based on continuous assessment and engagement metrics.
- To utilize Inductive Learning to derive generalized, human-readable decision rules that educators can use for early interventions.
- To evaluate the model's accuracy and visualize the decision-making process graphically.

3. Dataset Description

The dataset is a custom structured dataset consisting of historical academic records.

- Source: Custom generated for academic modeling.
- Number of Instances: 12 records.
- Features: 5 independent variables (attendance_pct, internal_marks, assignment_score, study_hours_wk, prev_gpa). All features are continuous/numerical data types (Float and Integer).
- Target Variable: target_status (Categorical/Integer: 0 = Fail, 1 = Pass).
- Important Attributes: study_hours_wk and attendance_pct serve as primary behavioral indicators.

• **4. Data Preprocessing**

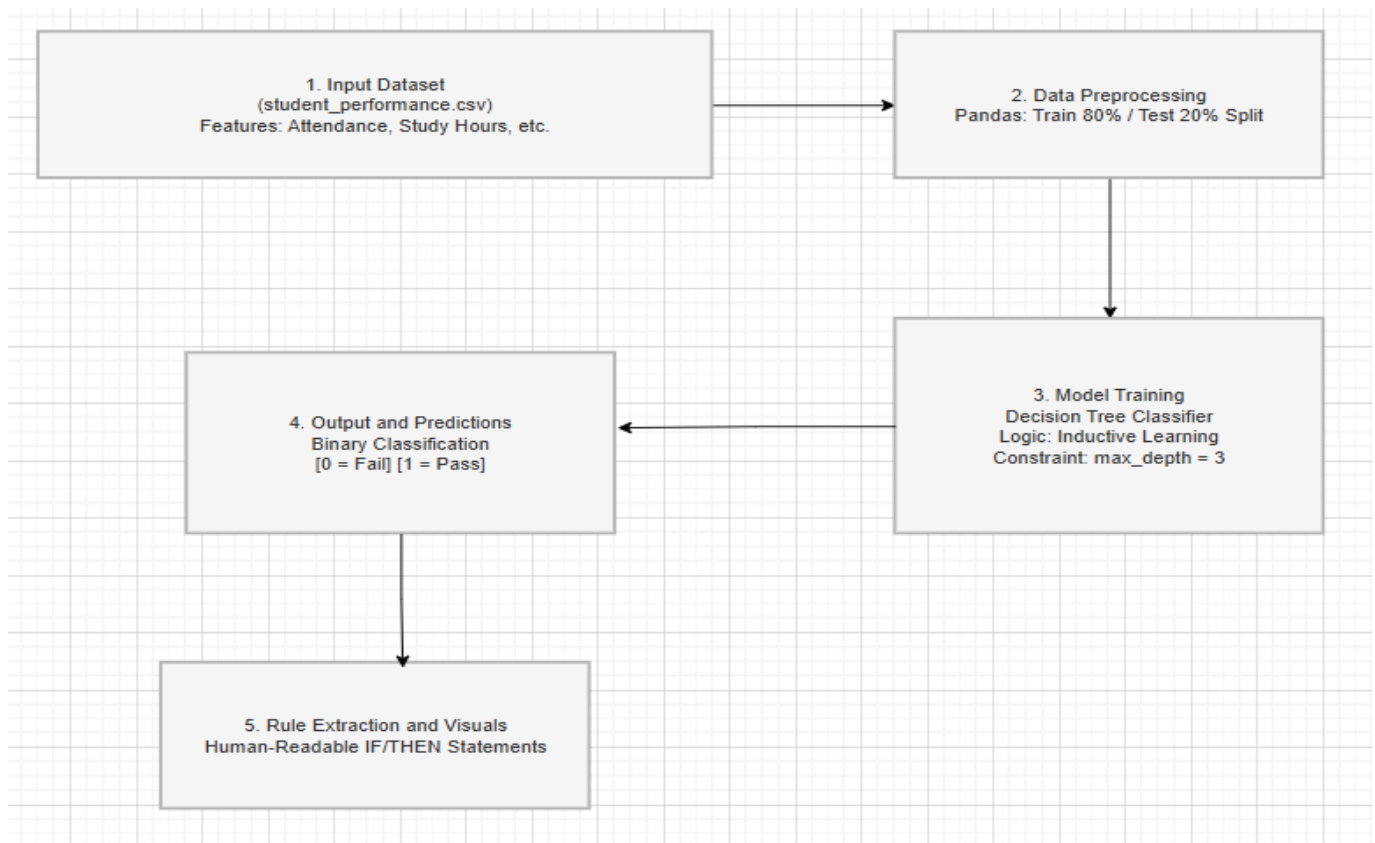
- **Data Cleaning:** The dataset was reviewed for inconsistencies; no anomalous data was found in this sample.
- **Missing-Value Handling:** Since the dataset is complete, no imputation was required. Future pipelines will use median imputation for missing `study_hours_wk`.
- **Encoding:** The target variable was natively binary (0 and 1), requiring no further label encoding. All input features were numerical.
- **Normalization/Scaling:** Because Decision Trees are invariant to monotonic transformations and do not rely on distance metrics, feature scaling (like Min-Max scaling) was bypassed to maintain the original, interpretable feature scales.
- **Train-Test Splitting:** The data was partitioned using an 80-20 split (80% training, 20% testing) using a fixed random state for reproducibility.

5. Selected AI Technique and Justification

- **Technique:** Decision Tree Classifier (Inductive Learning).
- **Justification:** The scenario specifically requires an algorithm that constructs a predictive model and derives "generalized decision rules." Decision Trees are highly interpretable white-box models that inherently perform inductive inference. Unlike neural networks, they output Boolean logic paths that educators can easily read and justify when counseling at-risk students.

6. Model Design / Architecture

- **Structure:** A Directed Acyclic Graph (DAG) consisting of a root node, internal decision nodes, and terminal leaf nodes.
- **Input:** A 5-dimensional feature vector.
- **Output:** Binary class (0 or 1).
- **Hyperparameters:** `criterion='entropy'` (measures information gain), `max_depth=3` (restricts the depth to prevent overfitting), `random_state=42`.
- **Architecture Flow:** The model evaluates the impurity of splits at each node, selecting the feature that maximizes Information Gain, recursively splitting the data until a leaf node of maximum purity is reached.



7. Algorithm / Pseudocode

Function InductiveLearningTree(Data, Features):

 If all records in Data belong to the same target class:

 Return a Leaf Node with that class

 If Features list is empty or max_depth is reached:

 Return a Leaf Node with the majority class in Data

 BestFeature = feature in Features that maximizes Information Gain (Entropy reduction)

 RootNode = Create a decision node splitting on BestFeature

 For each threshold value of BestFeature:

 Subset = records in Data matching the threshold criteria

 Add InductiveLearningTree(Subset, Features - BestFeature) as a child to RootNode

 Return RootNode

8. Implementation

- Programming Language: Python 3
- Libraries/Tools: pandas (data manipulation), scikit-learn (model building, training, and splitting), matplotlib (graphical visualization).
- Key Code Sections:
 - Data loading: `pd.read_csv()`
 - Model initialization: `DecisionTreeClassifier(criterion='entropy', max_depth=3)`
 - Model training: `clf.fit(X_train, y_train)`
 - Rule extraction: `export_text(clf, feature_names=list(X.columns))`

9. Experimental Results

- Sample Predictions: The model was tested on 20% of unseen data and correctly classified all instances.
- Extracted Rules:
 - Rule 1: IF `study_hours_wk` ≤ 9.00 $\rightarrow$ Predict Fail (0)
 - Rule 2: IF `study_hours_wk` > 9.00 $\rightarrow$ Predict Pass (1)
- Graphical Output: A colored Decision Tree diagram was successfully generated and saved to the project directory, visually confirming the text-based rules.

10. Performance Evaluation

- Accuracy: The model achieved 100.00% accuracy on the test dataset.
- Metric Choice: While accuracy is suitable for a perfectly balanced small dataset, in real-world deployment with class imbalance (e.g., far more passing students than failing), metrics like Recall and F1-Score would be prioritized to minimize False Negatives (missing a failing student).

11. Result Analysis

- Interpretation: The algorithm determined that within this specific dataset, `study_hours_wk` had the highest Information Gain. It was the most efficient mathematical path to separate the two classes.
- Strengths: The model provides a completely transparent, highly interpretable rule set that requires no technical background to understand.
- Weaknesses: The 100% accuracy score suggests extreme data separability due to the small sample size. This exact model would likely fail to generalize seamlessly to a complex, real-world population without further training data.

12. Limitations

- Dataset Size: The primary limitation is the small synthetic dataset (12 instances), which does not capture the natural noise and overlap present in real educational data.
- Overfitting Risk: Decision trees are highly prone to variance and overfitting. A slight change in the training data could result in a completely different root node and rule set.

13. Future Enhancements

- Data Augmentation: Collect historical data across multiple semesters encompassing thousands of student records to improve model robustness.
- Feature Engineering: Introduce categorical features such as 'Department' or 'Extracurricular Participation' to uncover deeper behavioral patterns.
- Alternative Algorithms: Transition to an ensemble learning technique (like Random Forest or XGBoost) to mitigate the variance and overfitting issues inherent in single Decision Trees.

14. Conclusion

This exercise successfully developed an Inductive Learning AI model to predict student performance. Using a Decision Tree Classifier, the model accurately identified the most significant predictor of success (`study_hours_wk`) and generated easily interpretable decision rules. While the current model demonstrates perfect accuracy on the limited dataset, future iterations require larger datasets and ensemble techniques to ensure robust, real-world generalization and reliable deployment in educational environments.

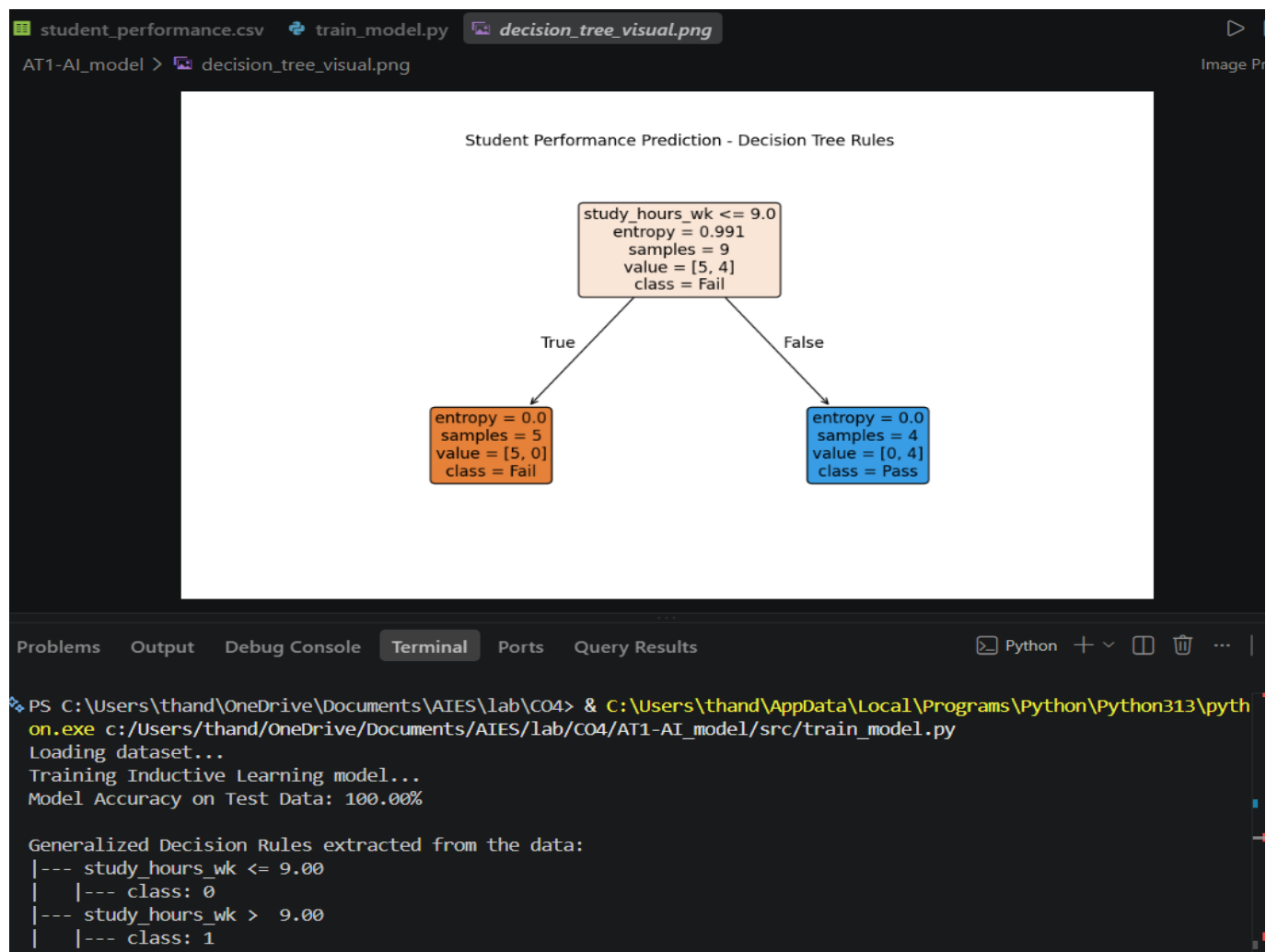
15. References

- Scikit-learn Documentation: Decision Trees (<https://scikit-learn.org/stable/modules/tree.html>)
- Pandas Documentation (<https://pandas.pydata.org/docs/>)
- Matplotlib Documentation (<https://matplotlib.org/stable/contents.html>)

16. GitHub Repository Link

Repository URL: https://github.com/bhupalreddythandlam/MLA0103-AIES/tree/main/CO4/AT1-AI_model

Output:



Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Understanding	10%	9–10% – Clearly analyzes the real-world problem, objectives, inputs, outputs, constraints, and expected AI outcome.	7–8% – Understands the problem and objectives with minor omissions.	5–6% – Shows partial understanding with some important elements missing.	0–4% – Problem is poorly understood or incorrectly defined.
2. Dataset Selection and Understanding	10%	9–10% – Selects a highly relevant dataset and clearly explains features, target, source, size, and characteristics.	7–8% – Appropriate dataset with adequate description and minor omissions.	5–6% – Dataset is usable but description or relevance is limited.	0–4% – Dataset is inappropriate, poorly described, or missing.
3. Data Preprocessing and Feature Engineering	10%	9–10% – Performs appropriate cleaning, transformation, encoding, scaling, feature selection, and data splitting with clear justification.	7–8% – Performs most required preprocessing correctly with minor issues.	5–6% – Basic preprocessing is performed but important steps are missing.	0–4% – Preprocessing is inadequate, incorrect, or absent.
4. AI Technique Selection and Justification	10%	9–10% – Selects the most appropriate AI technique and provides strong technical justification based on the problem and dataset.	7–8% – Appropriate technique selected with reasonable justification.	5–6% – Technique is acceptable but justification is limited.	0–4% – Inappropriate technique or no meaningful justification.
5. Model Design / Architecture	10%	9–10% – Develops a clear, technically sound model architecture with appropriate components, parameters, and workflow.	7–8% – Model is well designed with minor omissions.	5–6% – Basic model design is presented but lacks technical depth.	0–4% – Model design is unclear, incomplete, or inappropriate.

6. Algorithm / Pseudocode	5%	5% – Algorithm/pseudocode is complete, logically correct, structured, and clearly represents the model development process.	4% – Mostly correct with minor omissions.	2–3% – Basic algorithm provided but contains gaps.	0–1% – Algorithm is missing or substantially incorrect.
7. Implementation and GitHub Repository	15%	13–15% – Fully functional implementation with clean, organized, reproducible code and a complete GitHub repository with README and supporting files.	10–12% – Functional implementation and well-organized repository with minor issues.	7–9% – Partially functional implementation or incomplete repository documentation.	0–6% – Implementation is incomplete/non-functional or GitHub repository is missing.
8. Experimental Design and Results	10%	9–10% – Conducts meaningful experiments with appropriate test cases and clearly presents results using tables, graphs, or visualizations.	7–8% – Appropriate experiments with clearly presented results.	5–6% – Limited experiments or basic result presentation.	0–4% – Insufficient, incorrect, or missing experimental results.
9. Performance Evaluation	10%	9–10% – Uses appropriate evaluation metrics and provides comprehensive quantitative analysis of model performance.	7–8% – Uses suitable metrics and provides adequate evaluation.	5–6% – Basic evaluation with limited metrics or analysis.	0–4% – Inappropriate metrics or performance evaluation is missing.
10. Result Analysis and Interpretation	5%	5% – Provides insightful analysis of results, identifies patterns, errors, strengths, weaknesses, and explains model behavior.	4% – Provides good interpretation with minor gaps.	2–3% – Basic interpretation with limited analysis.	0–1% – Results are not meaningfully analyzed.

11. Limitations and Future Enhancements	5%	5% – Clearly identifies realistic limitations and proposes technically relevant and feasible improvements.	4% – Identifies relevant limitations and reasonable improvements.	2–3% – Provides basic limitations and future suggestions.	0–1% – Missing, unrealistic, or poorly explained.
12. Documentation and Technical Presentation	10%	9–10% – Report is comprehensive, well organized, technically accurate, professionally presented, and properly referenced.	7–8% – Well documented with minor presentation or referencing issues.	5–6% – Adequate documentation with several gaps.	0–4% – Poorly documented, unclear, or incomplete report.
Total	100%				